

Ingeniería de IA, Visión Artificial y Big Data aplicado a eSports

1. El Escenario del Problema

Las empresas líderes en el sector de los videojuegos y los eSports (como Riot Games, HoYoverse o Activision) utilizan Inteligencia Artificial para analizar en tiempo real miles de horas de transmisiones. Sus objetivos incluyen la detección de jugadas destacadas (*highlights*), el análisis del comportamiento de los jugadores y la identificación automática de errores de balance o trampas.

Vuestro grupo ha sido contratado como un equipo de **Ingeniería de IA** para diseñar un sistema de visión artificial capaz de identificar, a partir de un frame (captura de pantalla), el **videojuego** y el **estado de la partida**.

2. Objetivo del Reto

Diseñar, entrenar y desplegar una **Red Neuronal Convolucional (CNN)** profunda que procese un dataset masivo para clasificar imágenes en las siguientes categorías (ejemplos):

- **Videojuego:** *Valorant, Genshin Impact, Elden Ring, Cyberpunk 2077.*
- **Estado:** *Combate, Exploración, Menú de Inventario, Cinemática, Victoria/Derrota.*

3. Requisitos de Entrega (Transportabilidad)

Para que el proyecto sea profesional, debe ser **transportable**. Esto significa que debe poder ejecutarse en cualquier servidor (local, nube o aula) con un solo comando.

1. **Entorno Docker:** El proyecto debe vivir dentro de un contenedor.
2. **Modelo ONNX:** El resultado final debe exportarse a formato .onnx para su uso en aplicaciones multiplataforma.
3. **Gestión de Big Data:** Prohibido cargar datos con np.array. Debéis usar pipelines iterativos (tf.data).

4. Guía Paso a Paso

Paso 1: Acceso al "Cerebro" (Servidores)

Conectaos mediante SSH a los servidores potentes del aula. No intentéis entrenar esto en vuestros portátiles, ya que el volumen de datos colapsará vuestra memoria.

- **Comando:** `ssh usuario@IA-SERVER-XX`
- **Monitorización:** Usad `nvidia-smi` para vigilar la temperatura de la GPU.

Paso 2: El Pipeline de Datos (Big Data)

Dado que trabajamos con **80 GB de imágenes**, debéis configurar un generador que lea del disco "al vuelo".

- **Pista:** Usad `prefetch()` y `.cache()` para que la GPU nunca esté esperando al disco duro.

Paso 3: Diseño de la CNN

A diferencia del Perceptrón simple, aquí las entradas son tensores de (224, 224, 3)

- Utilizad capas `Conv2D` para detectar bordes y texturas.
- Añadid `MaxPooling2D` para reducir el tamaño y ahorrar cómputo.
- Implementad `Dropout` para evitar que la red memorice las capturas (Overfitting).

Paso 4: Uso de IA para guiarse

Podéis utilizar herramientas de IA generativa para:

- Depurar errores del Dockerfile.
- Ajustar hiperparámetros como el *Learning Rate*.
- Explicar el funcionamiento de capas complejas como las de *Global Average Pooling*.

5. Starter Kit: Pistas de Código

A. Dockerfile Base (Entorno Portable)

Este archivo asegura que la instalación sea automática y silenciosa (sin preguntas manuales).

```
FROM tensorflow/tensorflow:latest-gpu
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y libgl1-mesa-glx libglu1-mesa && rm -rf /var/lib/apt/lists/*
WORKDIR /app
RUN pip install --no-cache-dir opencv-python-headless tf2onnx scikit-learn
CMD ["python", "train.py"]
```

B. Pipeline Eficiente (tf.data)

```
# Carga iterativa para no saturar la RAM
train_ds = tf.keras.utils.image_dataset_from_directory(
    "/ruta/al/dataset/",
    image_size=(224, 224),
    batch_size=32
).prefetch(buffer_size=tf.data.AUTOTUNE)
```

6. Fuentes de Datos (Donde buscar)

Si no utilizáis el dataset precargado en el servidor, aquí tenéis bases de datos públicas de gran peso:

1. **Kaggle - Video Game Screenshot Dataset:** Miles de capturas etiquetadas por juego.
2. **YouTube-8M Dataset:** Frames extraídos de millones de vídeos de YouTube.
3. **Recopilación propia:** Podéis usar scripts de Python para extraer frames de transmisiones de Twitch guardadas.

7. Rúbrica de Evaluación

- **30% Ingeniería de Datos:** ¿Habéis gestionado los 80 GB sin errores de memoria?
- **30% Arquitectura:** ¿Es el modelo eficiente y preciso? (>90% Accuracy).
- **20% Portabilidad:** ¿El Dockerfile funciona a la primera?
- **20% Informe y Ética:** ¿Cómo habéis utilizado la IA para resolver el reto?

¡Ingenieros, el futuro de los eSports está en vuestra red neuronal!